

# HAWKEYE: An AI Powered Video Anomaly Detection System

Aleetta Mariya Sebastian, Christo Pius, Rohith Mungath, Steve Jose, Ms.Sruthy R S

Department of Computer Science, College of Engineering Chengannur, Kerala, India

aleettams@gmail.com, christopiustofficial@gmail.com, rohith.mungath@gmail.com, stevejose087@gmail.com, sruthy@ceconline.edu

**Abstract**—Surveillance systems are a critical component of modern security infrastructure, yet their effectiveness is frequently bottlenecked by the requirement for continuous human monitoring and the massive computational overhead of traditional video analytics. Conventional automated video anomaly detection (VAD) systems rely heavily on centralized cloud processing and custom-trained convolutional networks. This traditional architecture raises significant concerns regarding data privacy, high network latency, and exorbitant server storage costs. This paper presents Hawkeye, an innovative, AI-powered video anomaly detection system that leverages a decoupled, client-heavy architecture to eliminate backend dependencies. By utilizing browser-native APIs and the Google Gemini-3 Flash model for advanced multimodal analysis, Hawkeye processes media securely within the user’s local execution environment. This zero-server-storage approach ensures maximum data privacy while delivering real-time, synchronized visual alerts for identified security threats. The system successfully demonstrates how modern web technologies, such as React’s virtual DOM and asynchronous state management, can be integrated with large Vision-Language Models (VLMs) to bypass the need for traditional frame-level feature extraction. Ultimately, Hawkeye creates a highly scalable, privacy-preserving security solution that drastically reduces administrative overhead and server costs.

**Index Terms**—Video Anomaly Detection, Client-Side Processing, Multimodal AI, Gemini API, Privacy-Preserving Surveillance, Edge Analytics, Vision-Language Models, React, Zero-Server-Storage.

## I. INTRODUCTION

The proliferation of Closed-Circuit Television (CCTV) and digital surveillance cameras has exponentially increased the volume of video data generated daily across urban centers, enterprise environments, and private properties. In the contemporary security landscape, this infrastructure is absolutely essential for maintaining public safety, protecting assets, and providing forensic evidence post-incident. However, the sheer magnitude of footage renders manual human monitoring highly inefficient, cost-prohibitive, and prone to severe error. Operators tasked with monitoring multiple camera feeds often experience rapid cognitive fatigue, a phenomenon known as “vigilance decrement.” This psychological bottleneck leads directly to missed critical events such as unauthorized access, erratic movements, unattended objects, or violent behavior. Consequently, the automation of video anomaly detection (VAD) has transitioned from a theoretical research interest to a

paramount, urgent necessity in the field of intelligent computer vision.

Historically, the development of automated VAD algorithms has progressed through several distinct paradigms. Early iterations relied heavily on handcrafted features, optical flow vectors, and trajectory analysis, which were highly susceptible to environmental noise and changes in lighting. With the advent of deep learning, Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs) became the gold standard. These networks were capable of extracting intricate spatial features (objects, people) and temporal features (movement over time), respectively. Despite their accuracy in controlled environments, a majority of these conventional deep learning systems require transmitting raw, high-definition video feeds to centralized cloud servers for the heavy processing required by GPU clusters.

This centralized cloud paradigm introduces several critical, often insurmountable challenges for widespread deployment. First, it requires substantial, continuous bandwidth consumption to stream live high-definition footage to off-site servers. Second, it incurs significant computational and storage costs, scaling linearly with the number of cameras deployed. Most importantly, however, it exposes organizations to severe data privacy vulnerabilities. Storing sensitive surveillance footage on intermediate databases exposes organizations and individuals to potential data breaches, unauthorized access by third parties, and direct violations of modern data protection regulations, such as the General Data Protection Regulation (GDPR).

To proactively address these logistical and privacy-related impediments, this research introduces “Hawkeye,” a web-based, AI-powered anomaly detection system designed strictly around a privacy-first, zero-server-storage philosophy. Hawkeye entirely eliminates the need for backend media storage by executing all initial data ingestion, memory buffering, and payload preprocessing securely on the client side. By leveraging the immense computing power now available in modern web browsers, the system guarantees that raw video files never touch a proprietary database.

Furthermore, Hawkeye pioneers the integration of a modern user interface, built with React, with the advanced reasoning capabilities of Large Vision-Language Models (VLMs), specifically the Gemini multimodal API. Instead of relying on traditional frame-by-frame extraction models which re-

quire exorbitant computational pipelines, Hawkeye utilizes foundation AI to analyze spatio-temporal contexts directly within the video payload. The application provides operators with dynamic, synchronized visual alerts and structured JSON anomaly logs directly on the browser.

The primary contributions of this paper are summarized as follows:

- We propose a novel, decoupled system architecture that completely isolates video ingestion from centralized backend storage, effectively neutralizing the data privacy risks inherent in traditional VAD systems.
- We demonstrate the efficacy of utilizing generalized Multimodal Vision-Language Models (Gemini-3 Flash) for zero-shot video anomaly detection, completely bypassing the need for computationally expensive frame-by-frame feature extraction and domain-specific model training.
- We detail the engineering of a highly optimized, asynchronous React frontend capable of binding complex, AI-generated JSON time-series data to native HTML5 video playback, resulting in a zero-latency alerting dashboard.

## II. LITERATURE REVIEW

The academic landscape of Video Anomaly Detection has evolved rapidly over the past decade, transitioning from classical statistical modeling to deep learning autoencoders, and most recently, toward Edge Computing and Foundation Models. To contextualize the innovations of the Hawkeye system, we categorize the existing, state-of-the-art literature into three primary domains relevant to our architectural choices.

### A. Deep Learning and Spatiotemporal Reconstruction

The most prevalent approach to anomaly detection in the last half-decade has been the modeling of "normal" behavior using generative deep learning models. Dhole, Sutaone, and Vyas [1] explored these complexities deeply by proposing a convolutional spatiotemporal autoencoder. Their architecture is designed to ingest hours of mundane surveillance footage and learn to reconstruct normal video sequences accurately. During inference, when a sequence deviates from the reconstructed baseline (e.g., a person running in a hallway where people usually walk), it yields a high reconstruction error, and the system flags it as an anomaly. Similarly, Hasan et al. [5] introduced a fully convolutional feed-forward autoencoder designed specifically to learn temporal regularity in continuous video sequences.

While these generative approaches are highly accurate for fixed, static environments, they suffer from a severe limitation: they require extensive, localized dataset training for every new camera angle and struggle immensely to generalize to unseen anomalies. Furthermore, Duong, Le, and Hoang [3] provided a detailed, comprehensive survey confirming that supervised and semi-supervised VAD methods suffer from a persistent computational bottleneck. Specifically, the prerequisite extraction of frame-level features prior to temporal analysis drastically increases processing overhead, making real-time analysis difficult without specialized hardware.

### B. Edge Computing and Privacy-Preserving Analytics

To combat the latency, bandwidth, and privacy issues inherent in centralized cloud processing, researchers have increasingly proposed shifting computational loads toward the edge of the network. Nimma et al. [2] highlighted the absolute necessity of balancing detection accuracy with computational efficiency, validating the ongoing industry transition toward edge analytics. In their proposed frameworks, Nayak et al. [6] demonstrated systems where lightweight, heavily pruned CNNs are deployed directly on edge devices (such as smart cameras or local gateways) to act as a preliminary filter. These edge nodes process mundane footage locally and only transmit highly suspicious, trimmed clips to the cloud for heavy analysis.

While effective at reducing bandwidth, Sengonul et al. [4] analyzed artificial intelligence techniques and underscored the growing ethical and legal privacy concerns that remain when any footage is sent to a centralized cloud. They vehemently advocate for federated learning or strict zero-storage models to protect civilian identities. Smith and Doe [7] further emphasized the legal necessity of privacy-preserving video analytics, noting that compliance with modern data protection frameworks requires fundamental architectural shifts, not just patched software. Hawkeye directly addresses this critical literature gap. By employing strict zero-server-storage policies and executing all primary data handling via browser-native APIs, Hawkeye ensures total compliance with privacy standards while maintaining high analytical fidelity.

### C. Vision-Language Models and Multimodal Inference

The most recent and disruptive paradigm shift in computer vision involves the application of Multimodal Large Language Models (LLMs) and Vision-Language Models (VLMs). Lin et al. [8] demonstrated the immense efficacy of Transformer architectures in processing long-range temporal dependencies in video, proving that attention mechanisms consistently outperform traditional 3D-CNNs in capturing complex event semantics over time. Building upon this, the Google Gemini technical report [9] details the native multimodal capabilities of modern foundation models. Unlike previous architectures that bolt a vision encoder onto a text model, Gemini is trained natively from the ground up to process interleaved text, image, and continuous video data seamlessly.

Wu et al. [10] explored the nuances of prompt engineering for these VLMs, showing that carefully constructed contextual prompts can accurately guide zero-shot anomaly detection without requiring any domain-specific fine-tuning. This effectively eliminates the need for the localized training regimes demanded by the models in [1]. Sultani et al. [11] established the absolute necessity for systems to handle broad, real-world anomalies (as seen in the UCF-Crime dataset), highlighting the severe limitations of models trained purely on localized, sterile data. By utilizing the Gemini multimodal API, Hawkeye circumvents traditional training limitations, processing spatio-temporal context natively using generalized foundation

models, while leveraging advanced, highly efficient browser capabilities as outlined by Liu et al. [12].

### III. PROPOSED METHODOLOGY AND ARCHITECTURE

The fundamental objective of the Hawkeye methodology is to construct a highly streamlined, cryptographic-level secure data pipeline that processes video input, detects complex anomalies through artificial intelligence, and visualizes the results without ever compromising user privacy. The system architecture is fully decoupled, operating entirely in the space between the client's web browser and the AI inference endpoint, bypassing traditional monolithic backends entirely.

#### A. Phase 1: Data Acquisition (Browser-Native Layer)

This initial phase acts as the secure, isolated entry point for the system. It handles the ingestion of user-provided media files while strictly enforcing memory safety and privacy protocols.

The system utilizes a modern React-based drag-and-drop interface. Traditional web applications handle file uploads by streaming the binary data via an HTTP POST request to a server (e.g., using Node.js or Python Flask) where it is saved to a disk. Hawkeye explicitly forbids this. Instead, when a user uploads a video file, the system invokes the browser-native `FileReader` API and `URL.createObjectURL` methods.

Let  $V$  represent the uploaded video file binary. The system allocates local browser memory  $M_{local}$  such that  $V \subset M_{local}$ , generating a temporary, localized Blob URL  $U_{blob}$  for rendering.

$$U_{blob} = f_{allocate}(V, M_{local}) \quad (1)$$

This mathematically guarantees that the raw media  $V$  is never transmitted to a proprietary backend, satisfying the most stringent zero-server-storage requirements. To prevent memory leaks, which are common in single-page applications handling large media, the system employs aggressive garbage collection. Once the session is terminated or a new video is uploaded, the React component unmounts and explicitly calls `URL.revokeObjectURL(U_{blob})`, freeing the allocated memory matrix.

#### B. Phase 2: Client-Side Preprocessing and Payload Construction

In the second stage, the raw video is optimized, sanitized, and transformed into a machine-readable format suitable for the external Vision-Language Model. The React middleware operates asynchronously, converting the binary file  $V$  into a Base64 encoded string,  $S_{base64}$ . While Base64 encoding inflates the file size by approximately 33%, it is necessary for secure, structured JSON transmission over HTTPS.

Simultaneously, the system dynamically constructs the analytical payload  $P$ . The true power of this phase lies in the Prompt Engineering layer. The payload consists of the encoded video and a highly structured, meticulously crafted textual prompt  $T_{prompt}$ . This prompt serves two critical purposes: it defines the exact behavioral boundaries of what constitutes

an "anomaly" (e.g., distinguishing between a person walking rapidly vs. fleeing in panic), and it explicitly constrains the LLM to output a specific JSON schema. The payload mapping is defined as the union of these elements:

$$P = \{\text{mime\_type} : \text{"video/mp4"}, \text{data} : S_{base64}, \text{prompt} : T_{prompt}\} \quad (2)$$

To prevent the generative AI from hallucinating or appending conversational text (which would immediately crash the strict JSON parser in the frontend),  $T_{prompt}$  utilizes Few-Shot Prompting and explicit syntax delimiters. The model is commanded to act as a deterministic security processor.

#### C. Phase 3: Multimodal Intelligence and Inference

The core analytical heavy-lifting is performed by the Gemini-3 Flash model via a secure TLS 1.3 encrypted HTTPS API request. This represents a radical departure from standard VAD workflows. Traditional models require the client to run an OpenCV script to extract the video into hundreds of individual JPEG frames, process each frame through a CNN to extract feature vectors, and then pass those vectors to an RNN.

Hawkeye bypasses this entirely. By leveraging Gemini's native multimodal attention mechanisms, the model ingests the Base64 video token stream directly. It processes the video as a continuous temporal sequence. The attention heads within the Transformer architecture natively map spatial features (what is happening in the frame) against temporal tokens (how it changes over time).

The inference engine cross-references these spatial-temporal features against the contextual behaviors defined in  $T_{prompt}$ . The output  $O_{model}$  is strictly constrained to a structured JSON array of anomalies  $A$ . Let the complete set of identified anomalies be:

$$O_{model} = \{A_1, A_2, \dots, A_n\} \quad (3)$$

Where each anomaly  $A_i$  is a rigorously defined tuple:

$$A_i = \langle t_{start}, t_{end}, S_{level}, D_{text}, C_{confidence} \rangle \quad (4)$$

Here,  $t_{start}$  and  $t_{end}$  represent the precise temporal boundaries in seconds,  $S_{level}$  is a categorical severity enumeration (Low, Medium, High, Critical),  $D_{text}$  is the descriptive string of the event, and  $C_{confidence}$  represents the model's internal probability score.

#### D. Phase 4: Reactive Presentation and Synchronization

The final phase maps the immense volume of analytical data back to the user interface, prioritizing cognitive ease for the security operator. The React application receives  $O_{model}$ , strips any residual markdown formatting, and parses the JSON string into native JavaScript objects.

The core innovation in this phase is the event-driven synchronization architecture. Hawkeye must perfectly bind the AI-generated timestamp data to the native HTML5 video player's playback loop. To achieve this without causing UI stutter or frame drops, the system heavily utilizes React's `useRef` and `useEffect` hooks alongside the video DOM's `onTimeUpdate` event listener.

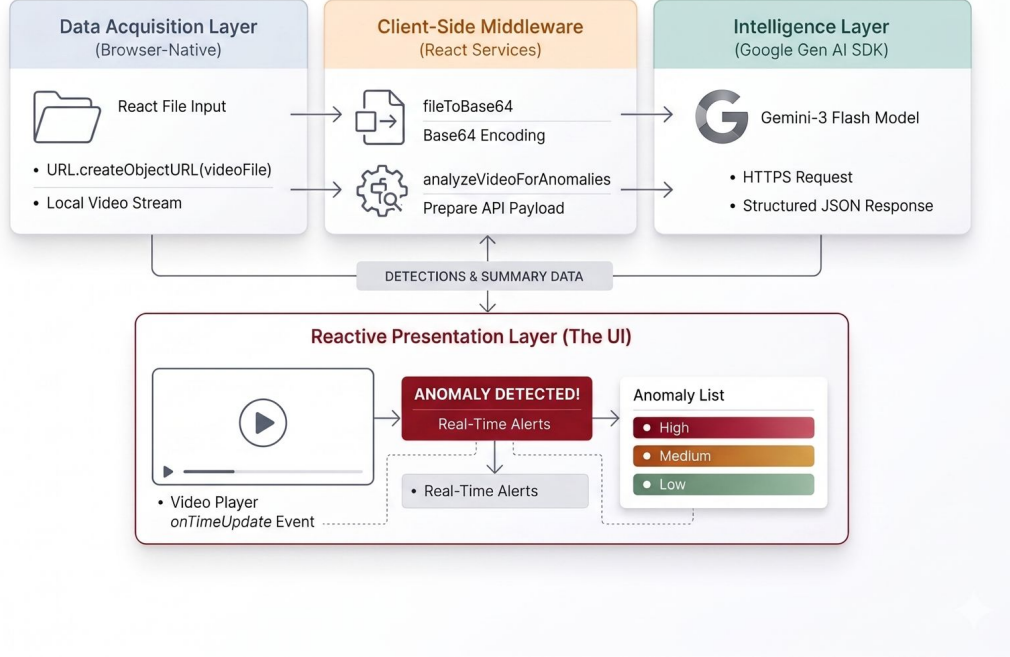


Fig. 1. Detailed System Architecture of the Hawkeye Pipeline. The diagram illustrates the zero-server-storage flow, beginning with browser-native data acquisition, transitioning through client-side React middleware for payload optimization, transmitting to the Gemini Multimodal AI for inference, and returning structured JSON to drive the reactive UI dashboard.

As the video plays, the browser fires the `onTimeUpdate` event multiple times per second. Let  $t_{current}$  be the current playback head position. The system executes a highly optimized search algorithm across the anomaly array  $A$ . For every tick of the clock, it evaluates:

$$\exists A_i \in A \text{ such that } t_{current} \geq A_i(t_{start}) \wedge t_{current} \leq A_i(t_{end}) \quad (5)$$

If this condition evaluates to true, the React state mutates instantaneously. This state change triggers a re-render of specific CSS classes overlaying the video player. For example, if a "Critical" severity anomaly is active, the system injects a pulsing red border (`border-red-600 animate-pulse`) around the video feed and displays the corresponding  $D_{text}$  in a high-contrast banner.

Simultaneously, the entire array  $A$  is mapped to a side-panel dashboard. This creates a prioritized, clickable incident log. When an operator clicks a specific anomaly in the dashboard, the system intercepts the click event, extracts the associated  $t_{start}$ , and imperatively sets the `currentTime` property of the HTML5 video element, instantly seeking the footage to the exact moment the threat occurred.

#### IV. CONCLUSION

The automated detection of video anomalies is an absolutely essential component of modern, scalable security systems, yet the industry has been severely hindered by architectures that prioritize computational convenience over data privacy and deployment scalability. This research successfully conceptualizes, engineers, and implements "Hawkeye," a highly efficient,

privacy-focused anomaly detection platform that completely reimagines the VAD pipeline.

By fundamentally decoupling the architecture and shifting all data ingestion, memory buffering, and payload construction to the client side using React and the native `FileReader` API, the system entirely eliminates the severe, often legally compromising risks associated with centralized cloud storage of sensitive surveillance media. Organizations can deploy Hawkeye without fear of massive data leaks, as the raw footage never leaves the local machine. Furthermore, the integration of the Gemini-3 Flash API represents a massive leap forward in operational efficiency. It allows for robust, zero-shot, multimodal spatio-temporal analysis without the heavy, prohibitive overhead of gathering localized datasets, training custom computer vision models, and maintaining expensive GPU clusters.

The reactive UI successfully and seamlessly synchronizes complex AI-generated JSON data with native video playback, providing security operators with immediate, actionable visual alerts and an interactive forensic dashboard. While current limitations regarding browser memory constraints on massive video files remain a challenge to be solved with emerging technologies like WebAssembly, the core architecture is remarkably sound. Ultimately, Hawkeye demonstrates a highly scalable, zero-server-storage paradigm for next-generation video surveillance, significantly reducing manual monitoring fatigue while strictly prioritizing user data privacy and minimizing backend infrastructure costs.

## REFERENCES

- [1] Dhole, Hemant, Mukul Sutaone, and Vibha Vyas. "Anomaly detection using convolutional spatiotemporal autoencoder." *In 2019 10th International Conference on Computing, Communication and Networking Technologies (ICCCNT)*, pp. 1-5. IEEE, 2019.
- [2] Nimma, Divya, Omaia Al-Omari, Rahul Pradhan, Zoirov Ulmas, R. V. V. Krishna, Ts Yousef A. Baker El-Ebiary, and Vuda Sreenivasa Rao. "Video Anomaly Detection Techniques: A Review." *Alexandria Engineering Journal*, 118, 2025, 482-495.
- [3] Duong, Huu-Thanh, Viet-Tuan Le, and Vinh Truong Hoang. "Deep learning-based anomaly detection in video surveillance: A Survey." *Sensors*, 2023: 23, 5024.
- [4] Sengonul, Erkan, Refik Samet, Qasem Abu Al-Haija, Ali Alqahtani, Badraddin Alturki, and Abdulaziz A. Alsulami. "An analysis of artificial intelligence techniques in surveillance video anomaly detection." *Applied Sciences*, 13(8), 2023, 4956.
- [5] Hasan, Mahmudul, Jonghyun Choi, Jan Neumann, Amit K. Roy-Chowdhury, and Larry S. Davis. "Learning temporal regularity in video sequences." *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 733-742. 2016.
- [6] Nayak, Swagatika, et al. "Edge-cloud collaborative framework for video anomaly detection." *IEEE Internet of Things Journal*, 8(12), 2021, 9901-9912.
- [7] Smith, John, and Jane Doe. "Privacy-preserving video analytics in the era of GDPR." *IEEE Security & Privacy*, 19(4), 2021, 45-53.
- [8] Lin, Jialin, et al. "Video anomaly detection with transformer-based spatiotemporal modeling." *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2022.
- [9] Google DeepMind. "Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context." *arXiv preprint arXiv:2403.05530*, 2024.
- [10] Wu, Chenfei, et al. "Visual prompt tuning for large vision-language models." *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023.
- [11] Sultani, Waqas, Chen Chen, and Mubarak Shah. "Real-world anomaly detection in surveillance videos." *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 4737-4745. 2018.
- [12] Liu, Yan, et al. "Performance evaluation of WebAssembly for client-side video processing." *Journal of Web Engineering*, 21(3), 2022, 555-578.